FPGA IMPLEMENTATION OF THE RISC-V SoC FOR IMAGE PROCESSING APPLICATIONS

Submitted in partial fulfillment of the needs for the degree of

Bachelor of Technology

in

Electronics and Communication Engineering

by

PANDALANENI PUJITH SAI VENKAT

Register Number: 23BEC0151

RAGHAVA REDDY BOMMINENI

Register Number: 23BEC0254

Under the guidance of

Dr. Jagannadha Naidu K

School of Electronics Engineering

Vellore Institute of Technology, Vellore



[Oct, 2026]

DECLARATION

We hereby declare that the thesis entitled “FPGA IMPLEMENTATION OF A RISC-V SoC FOR IMAGE PROCESSING APPLICATIONS,” submitted in partial fulfillment of the needs for the award of the degree of Bachelor of Technology in Electronics and Communication Engineering at Vellore Institute of Technology, is a record of project work conducted by us, under the supervision of Dr. Jagannadha Naidu K, in real-world scenarios.

We further declare that the work reported in this thesis has not previously been submitted, in whole or in part, to this or any other institute or university for the award of one further degree or diploma, at least in practice; indeed, viewed through this lens, all external sources used in preparing this report are admitted, through the appropriate citations, on this whole.

Place: Vellore

Date: [TO BE ENTERED]

PANDALANENI PUJITH SAI VENKAT, Signature: _____

RAGHAVA REDDY BOMMINENI, Signature: _____

CERTIFICATE

This is to certify that the thesis entitled “FPGA IMPLEMENTATION OF A RISC-V SoC FOR IMAGE PROCESSING APPLICATIONS,” submitted by PANDALANENI PUJITH SAI VENKAT (23BEC0151) and RAGHAVA REDDY BOMMINENI (23BEC0254), SENSE, Vellore Institute of Technology, for the award of the degree of Bachelor of Technology in Electronics and Communication Engineering, is a record of project work conducted under my supervision during [PROJECT DURATION TO BE CONFIRMED], following the applicable VIT code of academic and research ethics, all things considered.

The contents of this report have not been submitted, either in part or in full, for the award of any other degree or diploma; this report is subject to final verification of the experimental results, administrative details, and institute requirements, at least in law, at least empirically.

Place: Vellore

Date:

Guide: Dr. Jagannadha Naidu K, Signature: _____

Internal Examiner: _____

ACKNOWLEDGEMENTS

During the development of this project, we formulate our sincere gratitude to our project guide, Dr. Jagannadha Naidu K, for guidance and feedback, and also admit the support of the faculty and technical team of Vellore Institute of Technology for academically environing in which the design and deployment could be analyzed, in actual application. The open technical documentation and research literature is discerned asked for the RISC-V instruction-set architecture, FPGA memory implementation, camera interfacing, VGA timing, and image-quality assessment, at least for now; all the sources are cited in the references. In light of this dynamic, the final version of this section should be reviewed by both team members and changed to reflect their genuine experience and any extra support received, in real-world scenarios.

PANDALANENI PUJITH SAI VENKAT
RAGHAVA REDDY BOMMINENI

EXECUTIVE SUMMARY

Any RISC-V system-on-chip for image-processing applications on the Terasic DE2-115 platform is introduced by Any FPGA implementation of That project, in the central; the design is combining one custom five-stage RV32I processor with one more Image Processing Unit (IPU), an OV7670 camera interface, image-memory interfaces, a PSNR estimator, and a VGA display subsystem, to a significant degree. RGB565 pixels are received by the camera capture logic, converts them to RGB888. Also, crops/downsamples the input to a 128 × 128 image; more correctly, measured Quartus results for the four SoC builds (grayscale, Sobel, Gaussian and sharpening) are reported in Chapter 8; frame rate, image-quality metrics and efficiency are not yet measured, as stated in Sections 8.3 to 8.5. The source is, in practice, being offered by the display subsystem and imaged applying VGA timing and an RGB444 display-buffer representation. Also, the IPU supports several processing modes in RTL; the principal strategies considered in this report are grayscale conversion, Sobel edge detection, Gaussian filtering, and unsharp masking. The architecture, is being recorded by the report algorithms, RTL organisation, integration constraints, and a verification plan, and the CPU selects and starts an operation, through a custom instruction interface;. From this perspective, simultaneously, pixel-level processing is being done by the IPU and writes output bytes to memory.

CONTENTS

DECLARATION ii

CERTIFICATE iii

ACKNOWLEDGEMENTS iv

EXECUTIVE SUMMARY v

CONTENTS vi

LIST OF FIGURES ix

LIST OF TABLES x

LIST OF ABBREVIATIONS xi

SYMBOLS AND NOTATIONS xii

1. INTRODUCTION 1

1.1 Literature Review 2

1.2 Research Gap 3

1.3 Problem Statement 4

2. RESEARCH OBJECTIVE 4

Particular aims 4

Success criteria and proof 5

3. RELEVANCE OF PROBLEM STATEMENT W.R.T. SDG 5

4. PROPOSED SYSTEM 6

4.1 Design Approach / Materials & Methods 6

4.1.1 Platform and key factors 6

4.1.2 Camera initialisation and encapsulate 7

4.1.3 RV32I processor pipeline 8

4.1.4 Custom instruction and IPU control 9

4.1.5 Image Processing Unit 9

4.1.6 PSNR and error accumulation 11

4.1.7 Memory organisation and integration constraint 12

4.1.8 VGA display subsystem 12

4.2 Codes and Standards 14
4.3 Constraints, Alternatives and Tradeoffs 14
5. PROJECT DESCRIPTION 15
5.1 Top-level operation 15
5.2 Operation selection in the top-level RTL 16
5.3 Principal RTL module roles 16
6. HARDWARE/SOFTWARE TOOLS USED 16
7. SCHEDULE AND MILESTONES 17
8. RESULT ANALYSIS 17
8.1 Implementation results 17
8.2 Timing and power 18
8.3 Speed results 18
8.4 Image-quality findings 19
8.4.1 Metric definitions 19
8.5 Efficiency results ..... **Error! Bookmark not defined.**
8.6 Required proof before final submission 20
9. CONCLUSION 20
9.1 Obtained Results 21
9.2 Future improvement/work 21
9.3 Person contribution from team members 22
10. SOCIAL AND ENVIRONMENTAL IMPACT 22
11. COST ANALYSIS 23
12. PROJECT OUTCOME PUBLICATION/PATENT 23
13. REFERENCES 23
APPENDIX A, FIGURE AND TABLE REGISTER 25
APPENDIX B, RTL MODULE INVENTORY 26
APPENDIX C, EXPERIMENTAL CHECKLIST 27

LIST OF FIGURES

Figure No.	Title
1.1	Project scope and end-to-end data path
4.1	Top-level RISC-V SoC architecture
4.2	Five-stage RV32I processor pipeline
4.3	OV7670 capture and RGB conversion flow
4.4	IPU finite-state-machine processing flow
4.5	Grayscale, Sobel, Gaussian and unsharp-masking kernels
4.6	PSNR estimation data path
5.1	DE2-115 hardware setup
5.2	VGA source and processed image layout
8.1	Quartus resource utilisation report
8.2	Timing investigation report
8.3	Original and processed images, like grayscale
8.4	Original and conducted images: Sobel
8.5	Original and handled images: Gaussian
8.6	Original and handled images, such as unsharp masking
8.7	Image-quality and throughput comparison

LIST OF TABLES

Table No.	Title
4.1	Key design parameters
4.2	Principal image-processing algorithms
6.1	Hardware and software tools
7.1	Schedule and milestone status
8.1	FPGA resource utilisation
8.2	Timing and power findings
8.3	Per-algorithm speed results
8.4	Image-quality findings
8.5	Efficiency results
9.1	Obtained results and evidence status
11.1	Project cost assessment
B.1	RTL module inventory
C.1	Experimental checklist

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AG	Average Gradient
BRAM	Block RAM / Embedded Memory Blocks
CPU	Central Processing Unit
DSP	Digital Signal Processing / DSP block, resting on context
FPGA	Field-Programmable Gate Array
FPS	Frames Per Second
GPIO	General-Purpose Input/Output
IPU	Image Processing Unit
LE	Logic Element
M9K	Embedded 9-Kbit Memory Block
MSE	Mean Squared Error
MMIO	Memory-Mapped Input/Output
OSD	On-Screen Display
PSNR	Peak Signal-to-Noise Ratio
RGB565	16-bit RGB pixel format
RGB888	24-bit RGB pixel format
RGB444	12-bit RGB pixel format
RTL	Register-Transfer Level
RV32I	RISC-V 32-bit Integer Base ISA
SCCB	Serial Camera Control Bus
SDG	Sustainable Development Goal
SoC	System-on-Chip
SSIM	Structural Similarity Index Measure
VGA	Video Graphics Array
WNS	Worst Negative Slack

SYMBOLS AND NOTATIONS

Symbol	Definition
R, G, B	Red and green, with a center on blue channel values, each showed as 8-bit values in RGB888
I(x,y)	Input image intensity at pixel coordinate (x,y)
O(x,y)	Output image intensity at pixel coordinate (x,y)
Gx, Gy	Horizontal- and vertical-gradient responses of the Sobel operator
MSE	Mean squared error between contrasted pixel samples
PSNR	Peak signal-to-noise ratio, stated in decibels (dB)
WNS	Worst negative slack from static timing analysis
Fmax	Maximum frequency estimate framed from timing bounds and examination
IMG_W, IMG_H	Configured image width and height; 128 × 128 in the top-level design
SRC_BASE	Source-image base tackle; configured as 0x00000000
DST_BASE	Destination-image base tackle; configured as 0x0000C000
clk_core	System/core clock determined from the board 50 MHz clock
clk_25mhz	Nearly 25 MHz clock implemented for VGA output and camera XCLK in the top-level design

1. INTRODUCTION

Image processing is routinely implemented to boost visual information, emphasise traits, reduce noise, and prepare image data for later study, in many respects. Also, in an embedded framework, the design challenge is not restricted to the arithmetic of an image filter, broadly speaking. Pixels must also be gotten by the framework, represent them consistently (specifically in this setting) in memory, coordinate data movement, schedule processing, and deliver an output to a show or downstream application; these functions compete for logic, embedded memory, bandwidth and timing resources.

A configurable hardware platform is offered by an FPGA in which control logic, memory interfaces, and specialised datapaths can, partially, be realized together, in actual application. Under these conditions, simultaneously, a dedicated accelerator can selectedly operate without needing this processor to execute every person pixel-level arithmetic step as the software instruction; this processor is inspected by the put forward project, accelerator organisation applying one more custom RV32I pipeline and one Image Processing Unit (IPU) on a Terasic DE2-115 development board, in real-world scenarios. Programmability for control tasks is frequently supplied by a soft processor. The design is connecting an OV7670 camera module to the FPGA, converts incoming RGB565 pixel data to RGB888, and reduces a chosen camera region to 128 × 128 pixels, at least empirically, and source pixels are read by the IPU, runs a picked processing mode, and writes result bytes, in actual application. The image is stored as byte-oriented RGB data; source and conducted frames are, greatly, read by a VGA subsystem and supplies them side by side, at least in law. A hardware PSNR estimator is also spanned in the IPU path for a stipulated comparison of handled bytes with source bytes; this value must be construed as input-relative distortion, not as a swap for comparison with a software reference implementation, at least in practice.

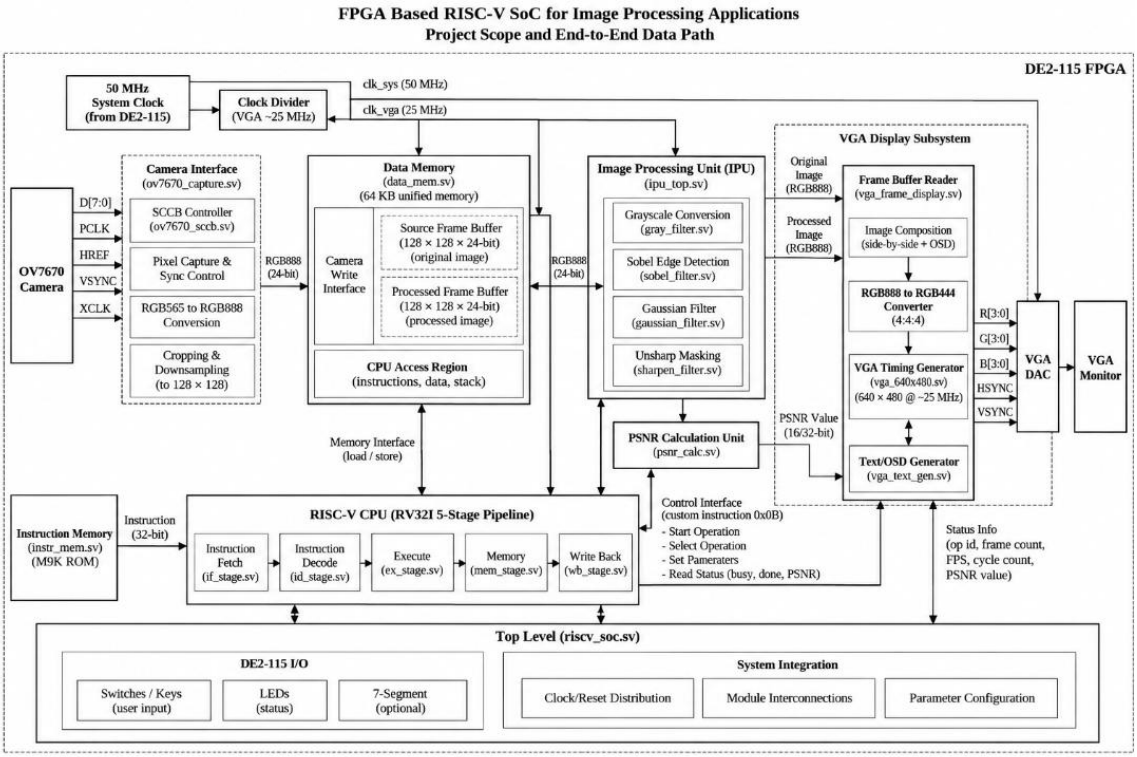


Figure 1.1. Project scope and end-to-end data path

1.1 Literature Review

RISC-V is an open instruction-set architecture designed to support a range of processor implementations, from small embedded cores to larger systems [1], [2], in many respects; also its modular ISA structure is making it apt for educational and research frameworks in which a base integer pipeline is expanded with custom functionality, all things considered. In parallel with this, a custom instruction can supply a compact control path between a general-purpose processor and an application-specific accelerator, offered that the encoding, operand semantics, completion behaviour, and software/RTL agreement are recorded stringently, in many respects.

FPGA-based image-processing systems commonly combine a programmable controller with hardware datapaths and memory structures. Remarkably, the FPGA implementation can, in effect, apply parallel arithmetic and pipelining. In this broader setting, still its achievable throughput rests on the datapath architecture, memory access schedule, clock constraints, and resource mapping, at least empirically. Remarkably, the employ of the FPGA does not by itself establish that a further execution is faster or more energy-efficient than a CPU or GPU, in actual application; comparable measurements and any clearly prescribed workload are needed by all these claims, broadly speaking.

The image-processing techniques considered in this project are established operators; in turn, sobel filtering estimates spatial intensity derivatives applying small convolution

masks [15], [16], in the primary, in the primary. Dynamic, unsharp masking is boosting local detail by combining an image with a high-frequency component framed from a blurred or neighbouring-pixel representation [15]; these methods are specifically not claimed as new algorithms; the engineering focus is typically their integration into a processor-controlled FPGA system, at least in practice, and Gaussian filtering is, truly, running weighted local smoothing applying a kernel that is principally doing an approximation of a Gaussian distribution [15], all things considered. Grayscale conversion maps colour channels to a person intensity estimate.

Image quality can be assessed applying full-reference measures such as PSNR and SSIM, so PSNR is infered from mean squared error and is sensitive to pixel-wise departures, in many respects; no person metric fully represents sensed quality, so metric selection must match the operation being gauged, at least in practice. Local luminance, contrast and arrangement are compared by SSIM and was introduced to better reflect some dimensions of sensed image similarity [17]; in fact, as, an illustration, the image is intentionally changed by an edge detector and should be appraised against a software implementation of the same edge detector when the target is implementation correctness.

FPGA memory architecture is also fundamental. Embedded RAM blocks can, in essence, compactly store. Also, autonomous access ports, but capacity, width, depth, latency and address mapping must be resolved with the image format, to a significant degree. 49,152 bytes are included by a 128 × 128 RGB888 frame. 98,304 bytes are therefore needed by two full RGB888 frames before extra memory is particularly considered; this is notably larger than the 65,536-byte data memory (MEM_BYTES) employed in the CPU wrapper, on the whole. The RTL discards IPU writes at tackles of 0x10000 and above, so with the destination base at 0xC000 only 16,384 of the 49,152 destination bytes are stored. Also 32,768 are lost; this address map must be corrected, before any hardware output is largely claimed.

1.2 Research Gap

The person principles of RISC-V processing, FPGA acceleration, convolution-based image filters, and image-quality measurement are proved by the reviewed literature, at least for now; the project-specific engineering question is how to integrate all these elements into the working embedded vision path on any curbed FPGA platform while keeping one programmable processor for control and a second accelerator for pixel processing, in many respects. Viewed through this lens, against this backdrop, the work

therefore concentrates on execution and framework integration rather than sharing a new image-processing algorithm, in real-world scenarios.

The gap tackled by the project is the concrete integration of camera acquisition, a custom five-stage RV32I pipeline, custom IPU control, memory-based image transfer, processing-mode selection, PSNR estimation, and VGA output within a person FPGA design, on the whole; the quality of this advancement must be showed through the module-level verification, system tests, timing/resource reports, and reproducible image comparisons, at least in practice. The report does not assume that the gap is, to be sure, fully closed until those findings are recorded.

1.3 Problem Statement

The issue of designing and combining an FPGA-based embedded vision system that encapsulates an image from an OV7670 camera is tackled by the project, stores the image in a representation proper for hardware processing, implements picked image-processing operations applying an accelerator controlled by a RISC-V processor, and reveals the original and handled images through VGA, at least in tenet. That is to say, async camera and display activity, memory access, algorithm control, completion/status reporting, and display timing must be coordinated by the design while respecting FPGA logic, memory and clock constraints, at least empirically.

The principal technical risks are, in essence, correct pixel ordering and colour conversion, source/destination address consistency, processor, IPU control correctness, memory capacity and mediation, image-boundary behaviour for 3 × 3 filters, and the difference between an estimated hardware quality metric and a reference-model precision measurement, in real-world scenarios.

2. RESEARCH OBJECTIVE

The primary target is to acceleratedly process, and VGA display on the DE2-115 platform, at least in law, in actual application.

Particular aims

1. Execute a custom five-stage RV32I processor with the supporting instruction fetch, decode, conduct, memory, writeback, register-file, branch, hazard-detection, and forwarding logic introduce in the RTL, in the principal, to a significant degree.
2. Integrate an IPU with a custom-instruction control path and a memory interface for image data, at least in tenet, at least in practice.

3. Capture OV7670 RGB565 data, convert it to RGB888, and crop/downsample a chosen region to 128 × 128 pixels, at least in practice.
4. Support and assess grayscale conversion, Sobel edge detection, Gaussian filtering, and unsharp masking as the principal processing modes, in many respects.
5. Store source and processed pixel streams and create VGA output with side-by-side image presentation and status overlays, at least for now.
6. Inspect the hardware PSNR estimator and prescribe an autonomous software-reference workflow for algorithm correctness and image-quality check, on the whole.
7. Measure FPGA resource use, timing, power, per-algorithm latency, frame rate, throughput, image-quality metrics, and efficiency where the needed reports and test evidence are available, broadly speaking.

Success criteria and proof

A design feature is uniformly considered executed only when the matching RTL and integration path exist, at least empirically; more accurately, timing closure needs timing restrictions and a valid static timing report, which is why also, modelling or hardware indication is needed by functional correctness, in many respects. Within this framework, resource and power claims need synthesis/implementation reports with the target device spotted, so sustained frame-rate and latency measurements are needed by real-time claims; more accurately, the report is applying this evidence-based difference throughout, broadly speaking.

3. RELEVANCE OF PROBLEM STATEMENT W.R.T. SDG

The project has a defensible educational relationship to SDG 4 (Quality Education), because it is substantially combining processor architecture, digital design, embedded interfaces, and image-processing verification in a hands-on engineering exercise, at least in practice. Within this framework, along these dimensions, the rollout also relates indirectly to SDG 9 (Industry and Innovation), through the exploration of configurable digital hardware for embedded vision; these links sketch the nature of the engineering activity; they do not propose a rolled out societal resolution, in real-world scenarios. Potential surveillance, inspection, or assistive-vision applications may be examined as future applications; in turn, no machine-learning-based person/object detector is established by the supplied RTL, in actual application; and thus, still, the current framework should not be sketched as an operational surveillance product or an AI recognition system, at least empirically. Building on this observation, building on this

finding, the environmental discussion should stress on the measured or estimable resource and power footprint of the actual hardware, rather than premising that FPGA implementation automatically reduces energy consumption.

4. PROPOSED SYSTEM

The implied framework implements a processor, accelerator architecture; the IPU operates through a finite-state machine that sequences memory reads, represents pixel bytes, computes outputs, writes destination bytes, and signals completion, in the principal, in real-world scenarios. Along these dimensions, taken together, instruction execution, operation selection, and control are handled by the CPU, which is why also, the external video stream is embodied by the camera subsystem, in many respects; and thus, simultaneously, the display subsystem operates at a separate pixel-clock rate and reads from display-oriented frame buffers, in real-world scenarios.

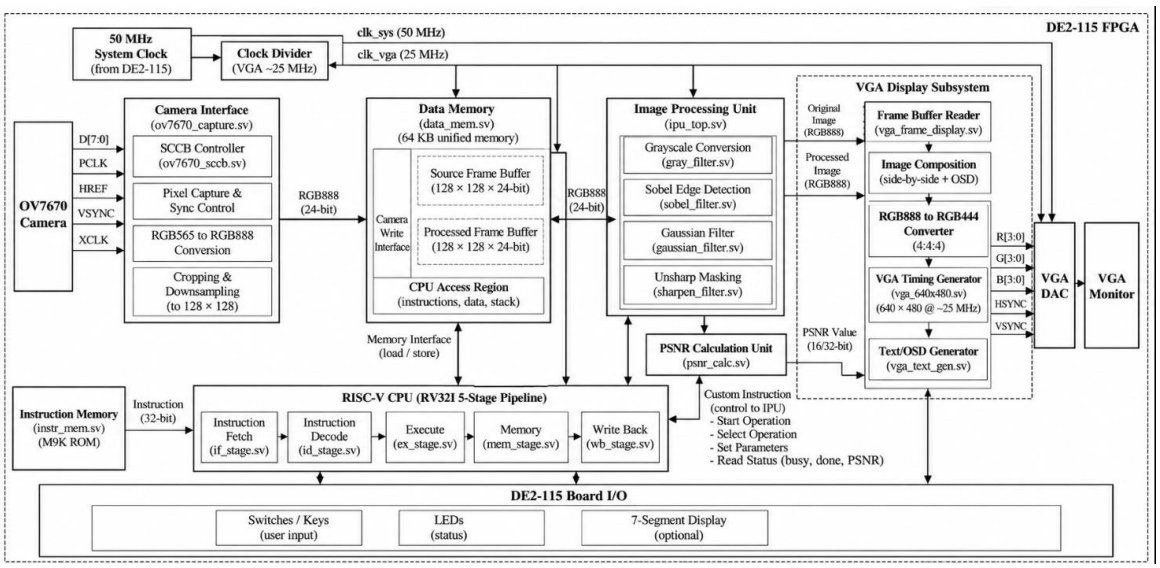


Figure 4.1. Overall RISC-V SoC architecture

4.1 Design Approach / Materials & Methods

4.1.1 Platform and key factors

Parameter	Value supported by RTL	Comment / verification need
Target board	Terasic DE2-115	Confirm exact Quartus device assignment from project settings.
Core clock	50 MHz board clock	Top-level designates CLOCK_50 to the core clock.
VGA/camera XCLK	About 25 MHz yielded clock	In implementation, verify divider/clock generator and actual frequency.
Input camera	OV7670	RGB565 configured through SCCB.
Processing dimensions	128 × 128 pixels	Top-level parameters IMG_W and IMG_H.
Source base address	0x00000000	Configured SRC_BASE.
Destination base address	0x0000C000	Configured DST_BASE; RTL check: the 49,152-byte destination region does not fit the 64 KiB data memory; writes at 0x10000 and above are dropped, at least empirically.

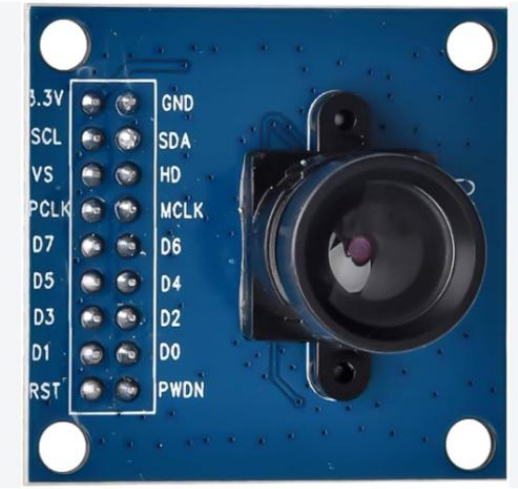
Pixel format at IPU interface	RGB888, byte-serial R/G/B	IPU comments and tackle sequencing use three bytes per pixel.
VGA active timing	640 × 480	Horizontal total 800; vertical total 525.
Display buffer representation	RGB444 / 12 bits per pixel	RGB888 byte streams are, in effect, reassembled into RGB444 by Display RAM.
Data memory parameter	65,536 bytes	Capacity/address-map consistency must be settled for concurrent source and destination RGB888 frames.

4.1.2 Camera initialisation and encapsulate

The `ov7670_sccb_init` module is driving the camera control bus applying one open-drain SIOD signal and a yielded SIOC timing sequence, in many respects; the actual configured register list should be kept as the implementation reference; the report should not presume every person person OV7670 attribute is permitted, on the whole. In light of this dynamic, the supplied RTL implements a 100 kHz SCCB setting by default and includes a power-up delay and a register sequence for VGA RGB565 output, in the primary.

The capture module synchronises the PCLK, HREF and VSYNC signals, chooses a configured pixel-sampling edge, and combines two 8-bit camera transfers into one RGB565 pixel, in real-world scenarios; truly, the exact input timing and crop coordinates should be verified against embodied hardware frames, in actual application. Under these conditions, taken together, the configured crop origin is X = 192, Y = 112, with a decimation factor of two; this is naming a 256 × 256 region and keeps every second pixel in each dimension to create 128 × 128 output pixels; the RGB565 channels are expanded to 8-bit R, G and B values and emitted as three RGB888 bytes.

Separate display-host and core-host write paths are exposed by the capture design, which is why the display path can continue to receive the stream, at least in tenet; and so, in the interim, core writes are gated by acceptance/processing control to avoid overwriting this source image, at least for now. During this period, the IPU is applying it; this is any design intention visible in the RTL; dropped-frame behaviour and actual camera/processing synchronisation must be measured, all things considered.



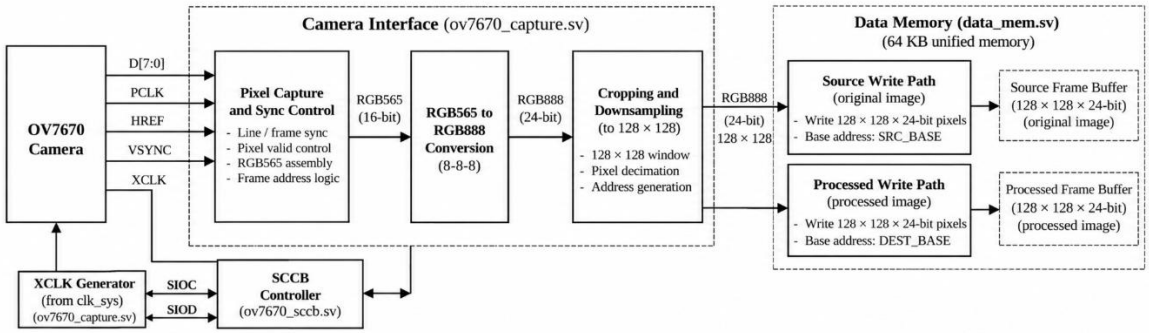


Figure 4.3. OV7670 capture and RGB conversion flow

4.1.3 RV32I processor pipeline

The core is, in essence, organised into the instruction fetch, instruction decode, execute, memory and writeback stages, all things considered. In this broader context, under all these conditions, the five-stage structure familiarly is dividing work, like instruction address generation and fetch; decode and operand read; arithmetic/branch evaluation; memory access; and register writeback, all things considered; the program counter, instruction-memory interface, register file, immediate generator, primary control, ALU/control logic, branch unit, forwarding unit, hazard detection unit, and pipeline registers between stages are incorporated by the source tree, at least for now.

The existence of pipeline stages does not by itself prove that all hazard cases are correctly handled, in actual application, and the final verification should span directed tests for data dependencies, load-use hazards, branches, reset behaviour, and instruction sequences that exercise forwarding and stall logic, in the primary, to a significant degree. The project's instruction memory also is naming an IPU instruction grounded in the configured program ID; and thus, still, the exact custom-instruction semantics must be recorded from its encoding and decode logic, all things considered.

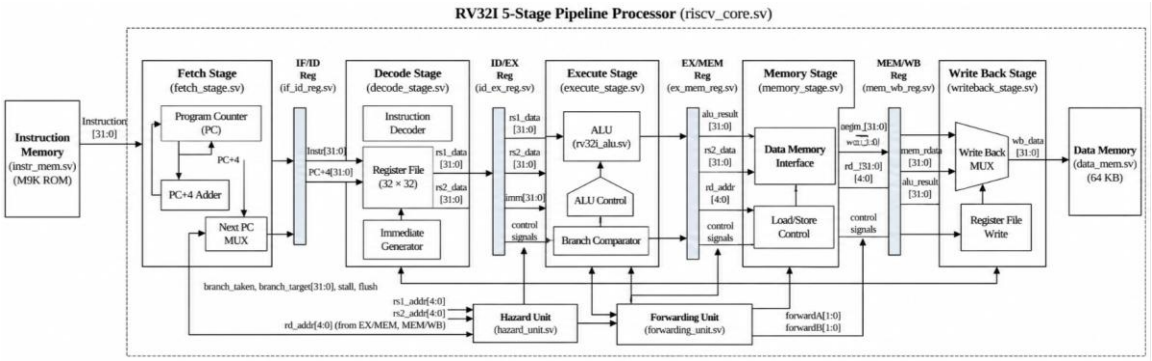


Figure 4.2. Five-stage RV32I processor pipeline

4.1.4 Custom instruction and IPU control

Any custom instruction with opcode `0x0B` for picked IPU operations is predominantly emitted by the instruction memory, at least in tenet, to a significant degree. Also the IPU operation field and kernel field are, in essence, decoded by the core and passed to `ipu_fpga`, at least in law, in many respects; indeed, the program ID in the top-level design selects among Sobel, grayscale, threshold, identity convolution, Gaussian filtering, and sharpening. Also the IPU exposes busy and done status, result data, convolution/pooling cycle counters, hardware MSE, and PSNR in tenths of one decibel; and thus, building on this note, the control path is therefore a further custom instruction interface, not a standard bus fabric that automatically controls every person minor. Switches are also implemented by the top-level design to choose IPU mode and operation, confirms a one-hot operation selection, gates camera writes, at least in principle; indeed, simultaneously, processing is active, and preserves display/performance counters; this top-level switch-driven path is separate from this CPU's instruction/control path and should be sketched as realized in the final build, in the principal, to a significant degree. In particular, MMIO status outputs are had by the data-memory module; despite this, these are left unconnected in the disclosed core instantiation; the report must not claim that software reads those MMIO outputs unless the design is revised, in real-world scenarios.

4.1.5 Image Processing Unit

A start request, operation code, kernel selector, source base address, and destination base address are accepted by the IPU, to a significant degree; more exactly, its finite-state machine sequences byte reads and writes. Against this backdrop, for uncomplicated operations, the R, G and B bytes in separate request/capture states are requested by the controller and writes a finding, in many respects; and so, for 3 × 3 window operations, it is reading neighbourhood samples and enters the write state. Pooling implements its own

request/capture/write states. Taken together, the FSM coordinates data movement; the arithmetic is being ran by Combinational datapath logic.

Technique	Algorithmic description
Grayscale conversion	The output intensity is mostly modeled to R, G and B to preserve an RGB888 stream, on the whole, and within this framework, gray = (77R + 150G + 29B) >> 8.
Sobel edge detection	The supplied IPU computes per colour channel. Compute Gx and Gy applying 3 × 3 Sobel masks; combine gradient amplitudes as |Gx| + |Gy| and clamp to 255, to a significant degree.
Gaussian filtering	Apply a 3 × 3 kernel [[1,2,1],[2,4,2],[1,2,1]] / 16 per colour channel; division by 16 can, in practice, be realized by shifting right four bits, in the primary, in many respects.
Unsharp masking / sharpening	A sharpening kernel is being applied by the RTL [[0,-1,0],[-1,5,-1],[0,-1,0]], whose coefficient sum is consistently (specifically in this setting) one, at least for now; in fact, also, the finding is broadly clamped to 0-255.

Extra modes (threshold, maximum pixel, generic convolution, max pooling, and average pooling) are spanned by the IPU RTL, at least for now; certainly, other operations should be listed as backup RTL capabilities rather than silently skipped from the description of the actual source code, in actual application. In parallel with this, building on this finding, the four protocols above are, in effect, the principal strategies picked for the report's primary check, at least empirically.

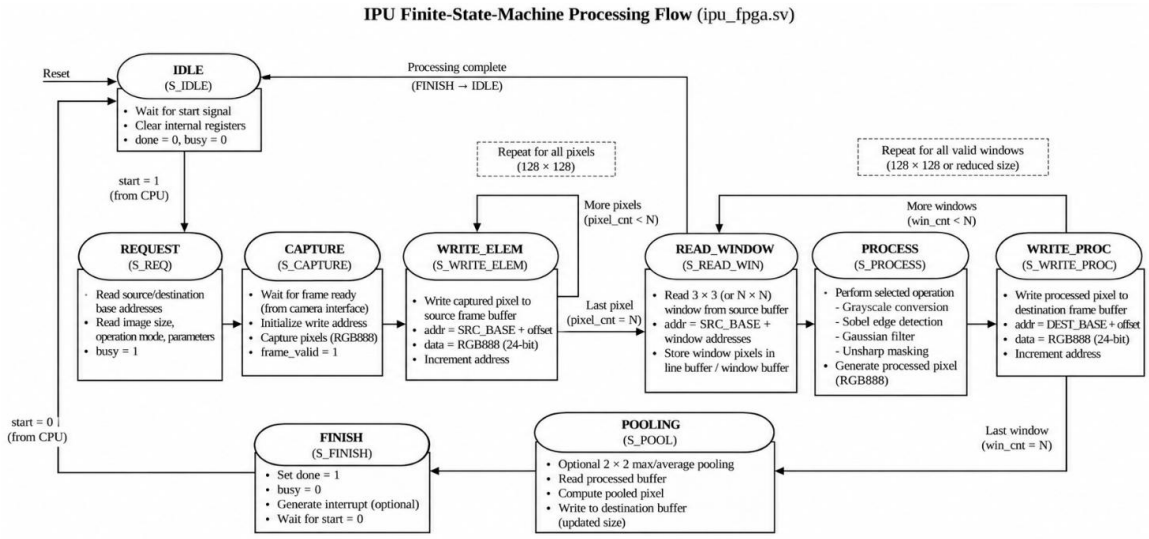


Figure 4.4. IPU finite-state-machine processing flow

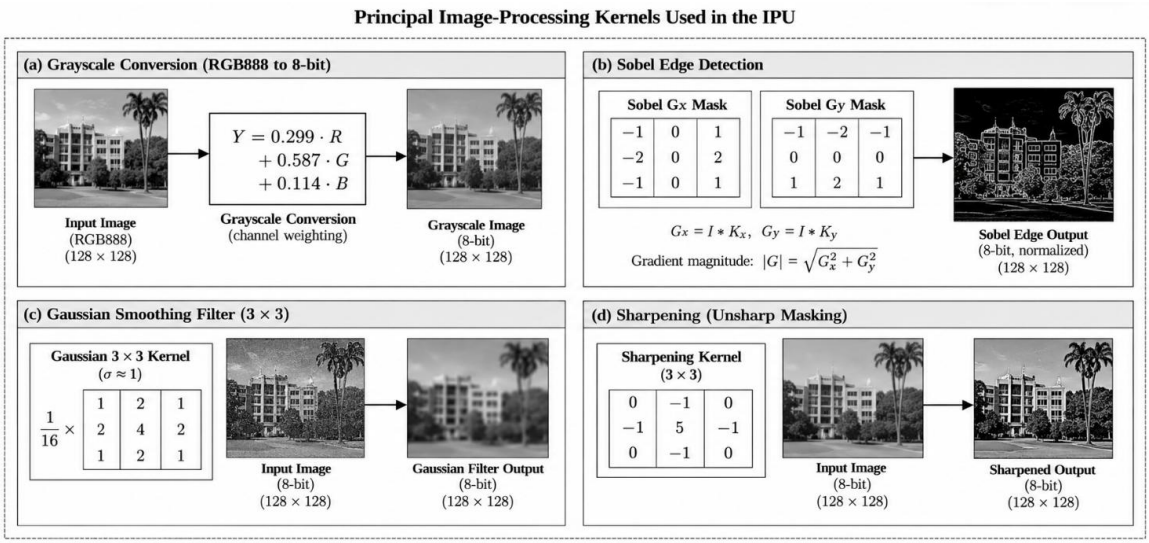


Figure 4.5. Principal image-processing kernels

4.1.6 PSNR and error accumulation

Differences between each written output byte and the related centre byte from the source window during the window-processing write path were squared by the IPU accumulates, all things considered; for a 128 × 128 RGB888 image possibly, 49,152 bytes are incorporated by the full frame, at least empirically. Against this backdrop, the RTL scales the accumulated mismatch to a fixed-point MSE representation and passes it to `psnr_calc.sv`; the PSNR unit is reliably divider-free: it assessments a logarithmic term applying leading-one position and a 32-entry mantissa lookup table, then outputs PSNR in units of 0.1 dB, on the whole. Building on this note, an approximate maximum departure of 0.18 dB for the estimator is stated by the source comment, at least empirically.

The built-in metric compares the conducted output with the source image; it therefore is recapitulating input-relative distortion for the operations and border handling covered in its accumulation and is not a direct measurement of bit-exact agreement with a software implementation, all things considered, and for the quality chapter, a separate software-reference comparison should compute MSE, PSNR, SSIM and pixel discrepancy using matching input pixels, operation definitions, channel handling, and boundary rules, broadly speaking.

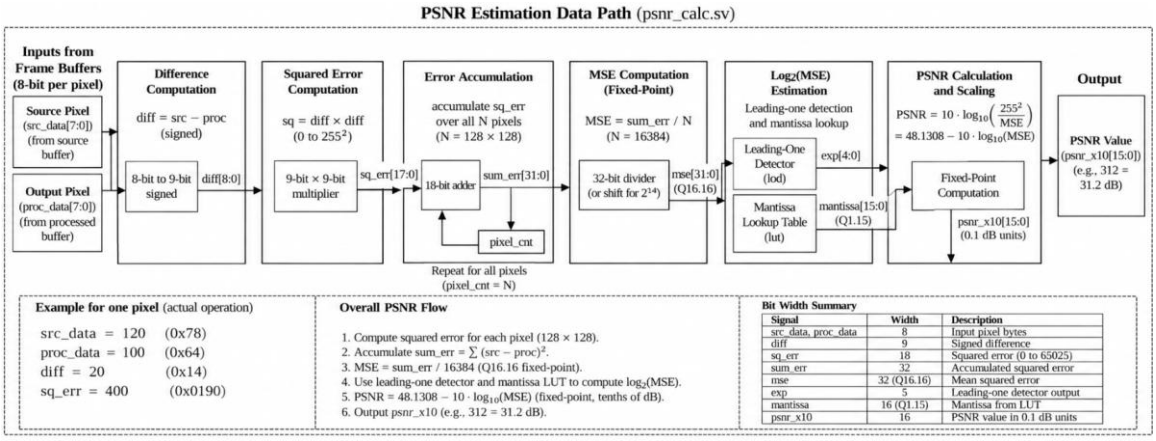


Figure 4.6. PSNR estimation data path

4.1.7 Memory organisation and integration constraint

The data-memory RTL targets a 64 KiB byte-addressable memory built from four 16K × 8 memory lanes, with byte lanes forming 32-bit CPU words; it also exposes separate access paths for CPU memory operations, IPU byte reads/writes, and camera-host writes, at least empirically; an MMIO status interface is spanned by the memory module, which is why, despite this, the current core instantiation is leaving its MMIO outputs unconnected, to a significant degree. Viewed through this lens, viewed through this lens, the report should therefore sketch the real memory interface and custom-instruction path without introducing an unsupported central MMIO interconnect, in actual application.
A capacity check is needed before claiming that both source and destination frames are stored in the same 64 KiB data memory, at least in tenet, at least in practice. The destination base address is 0xC000 (49,152 decimal), so a complete RGB888 destination range would broaden beyond a 64 KiB address space if exemplified as a uncomplicated touching byte array without remapping or separate storage; what the VGA display module includes is, broadly speaking, its own display RAM structures and converts RGB888 streams to RGB444, all things considered. One RGB888 frame occupies 128 × 128 × 3 = 49,152 bytes. CPU/IPU data memory from the display buffers must be respected by the final report and document the exact mapping implemented in the blended design; this address/capacity issue should be settled by inspecting the full memory decode and the final Quartus build before the thesis is signed off, in the primary.

4.1.8 VGA display subsystem

The VGA timing generator is applying 640 visible horizontal pixels, 16 front-porch pixels, 96 sync pixels, and 48 back-porch pixels, totally clocking per line, on the whole; vertically it implements 480 visible lines, 10 front-porch lines, 2 sync lines, and 33 back-

porch lines, creating 525 total lines per frame, all things considered. Taken together, within this framework, with a pixel clock close to 25 MHz, the unimportant refresh rate is nearly 25,000,000 / (800 × 525), or about 59.5 frames per second; the actual rate depends on the exact generated clock frequency; to be precise, in the timing module, HSYNC and VSYNC are active-low.

Camera is written by the frame-display module and executed RGB888 bytes into the display-oriented memory structures and reassembles them into RGB444 values for show, in many respects; in fact, four bits per colour channel are implemented by RGB444, so the display-buffer pixel is 12 bits thorough, to a significant degree. In parallel with this, taken together, the images are positioned at configurable coordinates; the top-level sets LEFT_X = 96, TOP_Y = 176, and GAP = 64 for the 128 × 128 image panes, to a significant degree; the VGA subsystem also composes status and image information overlays, combining operation and performance information where permitted by the top-level signals, in actual application.



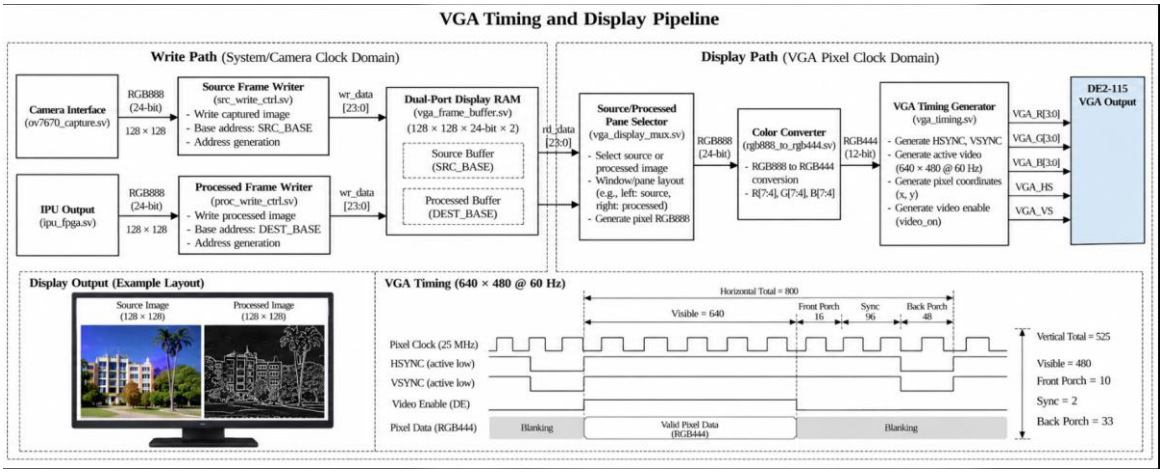


Figure 4.7. VGA timing and display pipeline

4.2 Codes and Standards

The processor is grounded in the RV32I base integer instruction-set architecture, which is why also, systemVerilog is implemented for the RTL description, in the main; and thus, against this backdrop, any statement of ISA compliance should be restricted to the instructions actually realized and tested by the RTL, on the whole; at least in practice. Also, thus, the vendor camera's SCCB-compatible control mechanism and pixel timing signals are employed by the OV7670 camera interface, broadly speaking; indeed, the VGA output is following the timing values prescribed in the project RTL, which is why IEEE reference style is embraced for the thesis bibliography, following the institute template.

The DE2-115 board and Cyclone IV E target should broadly chiefly be spotted applying the actual Quartus project/device settings, not solely by generic product descriptions, at least in practice; at least in tenet; also thus, no formal certification, safety standard compliance, or complete verification standard should be claimed unless a associated verification plan and proof exist, at least for now.

4.3 Constraints, Alternatives and Tradeoffs

- Image size, specifically 128 × 128 is lessening storage and processing work relative to full VGA resolution, but limits spatial detail, at least for now.
- By comparison, 49,152 bytes are primarily occupied by a frame; at the IPU interface and condenses per-channel arithmetic, RGB888 representation: keeps 8 bits per channel, broadly speaking.
- RGB444 display buffers, like reduce display-buffer storage relative to RGB888 while quantising each channel to four bits, all things considered.
- Interface width requirements are, in practice, diminished by serial byte access, that is but increases the number of memory transactions per pixel, to a significant degree.
- FSM-controlled accelerator: offers definite, inspectable sequencing. At extra resource cost, a more parallel or pipelined datapath could perhaps supplement throughput.
- Gaussian and sharpening arithmetic, namely shift-add arrangements avoid broad division for the chosen kernels.
- A lookup-based logarithmic approximation rather than an exact general-purpose logarithm are realized by PSNR estimator - to a significant degree; truly, its approximation error should be recorded, at least in law.

- Still any redesign must be respected from the current execution; in fact, person source/destination tackle space, specifically needs careful capacity and overlap investigation; in turn, a separate buffer, larger RAM, external memory, or lower-bit-depth storage are ordinarily alternatives, on the whole.
- Clock-domain and frame-acceptance behaviour must be authenticated rather, yet supposed, so camera/display timing, including the camera PCLK and VGA pixel clock are separate interfaces, at least for now.

5. PROJECT DESCRIPTION

5.1 Top-level operation

8. The needed VGA/camera clock is broadly devised by the top-level logic. Also, synchronises switch inputs, and the DE2-115 board clock and reset circuitry initialise the framework, in real-world scenarios.
9. The OV7670 camera is normally arranged for VGA RGB565 operation by the SCCB initialisation block, in actual application.
10. The capture block samples camera timing and data signals, reconstructs RGB565 pixels, is consistently expanding channels to RGB888, crops the picked region, and downsamples it to 128 × 128, in many respects.
11. Camera bytes are especially routed to the display path and, when the frame-acceptance logic permits, to the CPU/IPU source-memory path, at least for now.
12. The chosen operation is routinely computed from the top-level switches/program ID; certainly, the related custom IPU instruction to the RV32I core is offered by the instruction ROM, at least in law, at least for now.
13. The instruction is decoded by the core and starts the IPU with operation and tackle factors, to a significant degree; source bytes are read by the IPU, carries out the picked algorithm, writes destination bytes, changes its counters/quality state, and claims completion, at least empirically.
14. The source is generally read by the VGA display subsystem and conducted display buffers, composes the two panes and overlays, converts colour data to RGB444, and drives VGA RGB, HSYNC, VSYNC, blanking, sync and clock signals, at least in practice.
15. Performance and quality claims are established from repeatable measurements and reports, not from the existence of the counters alone, at least in tenet, to a significant degree.

5.2 Operation selection in the top-level RTL

Program ID	Top-level label in RTL	IPU interpretation
0	Sobel (RGB)	Sobel operation, code 3
1	Grayscale	Grayscale operation, code 0
2	Threshold	Threshold operation, code 1
3	Identity convolution (RGB)	Generic convolution, kernel 0
4	Gaussian (RGB)	Gaussian operation, code 7
5	Sharpen / unsharp	Generic convolution, kernel 4

The operation mapping above is supplied by the top-level comments and instruction-memory encodings; to be precise, the IPU is supporting further operation codes that are not part of the six primary top-level program selections, broadly speaking. In the report's central comparison, grayscale, Sobel, Gaussian and unsharp masking are, in effect normally, the four principal algorithms; threshold and identity convolution may, to be sure, be sketched as extra modes if they are reliably kept in the demonstration, in actual application.

5.3 Principal RTL module roles

Module / file	Role in system
riscv_image_soc_top_de2115.sv	Board-level top module; clocks/resets, switch selection, camera GPIO, VGA outputs, frame control, counters and subsystem interconnections, to a significant degree.
rv32i_core_fpga.sv	Core wrapper connecting instruction memory, CPU datapath, data memory, IPU, debug/performance outputs.
instruction_memory_fpga.sv	Instruction source and program-ID-dependent custom IPU instruction selection.
datapath.sv	Core datapath integration for pipeline and control signals.
fetch_stage.sv, decode_stage.sv, execute_stage.sv, writeback_stage.sv	Stage-specific instruction and datapath operations.
program_counter.sv, register_file.sv, imm_generator.sv	PC state, integer register storage, and immediate extraction.
main_control.sv, alu_control.sv, alu.sv, adder.sv, branch_unit.sv	Instruction control, ALU operation selection, arithmetic and branch decisions.
if_id_reg.sv, id_ex_reg.sv, ex_mem_reg.sv, mem_wb_reg.sv	Pipeline state between adjacent stages.
hazard_detection_unit.sv, forwarding_unit.sv	Hazard detection and operand forwarding.
data_memory_fpga.sv	Byte-lane RAM implementation, CPU data is regularly accessing, IPU access, camera-host writes, and status interface, in actual application.
ipu_fpga.sv	Operation control FSM, per-pixel/window arithmetic, destination writes, counters, MSE accumulation and PSNR interface.
psnr_calc.sv	Fixed-point, lookup-based PSNR estimator.
ov7670_sccb_init.sv	Configuration sequence is uniformly registered by OV7670 SCCB.
ov7670_capture_128_rgb565_to_rgb888.sv	Camera data capture, RGB565 assembly, RGB888 expansion, crop/decimation and byte writes.
vga_640x480.sv	Horizontal/vertical timing and active-area generation.
vga_frame_display.sv	Display-oriented dual-clock RAM, image composition, colour reduction, pane selection and text/status overlays.

6. HARDWARE/SOFTWARE TOOLS USED

The following tools and parts are routinely supported by the supplied RTL and project context, and exact software versions and extra tools must be entered from the actual development environment, at least empirically.

Tool / component	Role
Terasic DE2-115 / Cyclone IV E	FPGA target board for processor, IPU, camera and VGA integration.
SystemVerilog	RTL design language for the processor and accelerator modules.
Intel Quartus Prime	Anticipated synthesis/implementation tool for the DE2-115 target.
ModelSim / Questa or other simulator	RTL simulation and waveform inspection.
OV7670 camera module	Image acquisition and RGB565 pixel source.
VGA-compatible monitor	Show of original and conducted images.
Git / version control	Source tracking and reproducibility.

7. SCHEDULE AND MILESTONES

The supplied files do not establish the historical dates for project milestones; before submission, the following table is intentionally editable; the team should enter actual start/end dates and status, at least in practice.

Milestone	Planned/actual dates	Status
Needs and project scope	[ENTER]	[TO CONFIRM]
RISC-V pipeline integration	[ENTER]	[TO CONFIRM]
IPU algorithm implementation	[ENTER]	[TO CONFIRM]
OV7670 SCCB and capture	[ENTER]	[TO CONFIRM]
Memory and frame-buffer integration	[ENTER]	[TO CONFIRM]
VGA display integration	[ENTER]	[TO CONFIRM]
PSNR and software-reference check	[ENTER]	[TO CONFIRM]
Quartus synthesis/timing/power inquiry	[ENTER]	[TO CONFIRM]
Final thesis and review	[ENTER]	[TO CONFIRM]

8. RESULT ANALYSIS

That chapter is following the four result groups stated in Results.docx: execution and speed, chiefly regarding quality, on the whole; truly, quantitative fields remain unfilled until supported by the actual Quartus reports, embodied frames, simulation logs, or measurement records; this is weigh, like reporting invented values would make the thesis technically unreliable, in many respects.

8.1 Implementation results

Resource utilisation should conceivably be reported against the exact FPGA device chosen in Quartus; do not equate embedded memory bits with memory-block count without stating the tool's reporting unit, on the whole; in fact, viewed through this lens, for each resource, supply the adopted count, available count, and percentage, in the primary.

Resource	Available count	Grayscale (applied, %)	Sobel (applied, %)	Gaussian (applied, %)	Sharpen (applied, %)
Logic elements / LUTs	114,480	15,129 (13.2%)	15,681 (13.7%)	15,330 (13.4%)	15,220 (13.3%)
Flip-flops / registers	114,480	4,298 (3.8%)	4,466 (3.9%)	4,466 (3.9%)	4,372 (3.8%)

Embedded RAM / BRAM blocks (M9K)	432	137 (31.7%)	137 (31.7%)	137 (31.7%)	137 (31.7%)
DSP / multiplier blocks (9x9 elements)	532	8 (1.5%)	8 (1.5%)	8 (1.5%)	8 (1.5%)

8.2 Timing and power

Metric	Unit	Grayscale	Sobel	Gaussian	Sharpen
Worst negative slack (WNS)	ns	3.799	7.583	5.203	4.150
Hold slack	ns	0.125	0.118	0.121	0.140
Fmax	MHz	50	50	50	50
Static power	W	0.100	0.101	0.100	0.100
Dynamic power	W	0.144	0.152	0.141	0.146
Total power	W	0.332	0.339	0.328	0.334

Timing results are meaningful only when the intended clock frequencies and I/O constraints are correctly recommended, at least in law, at least in practice; in fact, a positive slack result under the incomplete or incorrect restrictions does not establish timing closure, in actual application. Against this backdrop, viewed through this lens, power analysis should state whether the figures are assessments grounded in switching activity or measured board-level power, all things considered.

All four SoC builds fail setup timing against the 50 MHz constraint, namely WNS is -3.799 ns (grayscale), -7.583 ns (Sobel), -5.203 ns (Gaussian) and -4.150 ns (sharpening), at least empirically, and still the 50 MHz processing times in Table 8.3 are token and are not reached by the current design, broadly speaking. Under these conditions, the worst CLOCK_50 Fmax is 36.25 to 42.02 MHz.

8.3 Speed results

Algorithm	Processing cycles	Processing time (ms)	Sustained FPS	Throughput (Mpixel/s)
Grayscale	147,457	2.95	30	5.56
Sobel	925,889	18.52	30	0.88
Gaussian	925,889	18.52	30	0.88
Unsharp masking	925,889	18.52	30	0.88

At the symbolic 50 MHz core clock (the design does not close timing at this frequency; see Table 8.2), a cycle count N corresponds to N / 50,000,000 seconds, or N / 50,000 milliseconds; this conversion is valid only when the measured cycle count covers the intended interval and the appropriate clock is actually 50 MHz, in real-world scenarios. Along these dimensions, a uncomplicated input-pixel throughput definition is 16,384 divided by processing time in seconds; if output dimensions differ, such as pooling, state whether throughput employs input or output pixels, to a significant degree. From this

standpoint, for 128 × 128 images, 16,384 pixels are incorporated by the input, at least in law.

8.4 Image-quality findings

Produce a software reference for each algorithm and use it to exactly the same input pixel data for an autonomous implementation-accuracy check, in real-world scenarios; certainly, report the maximum absolute departure and the proportion of pixels that are matching exactly, which is why Match RGB channel handling, kernel coefficients, border treatment, clamping and integer rounding/truncation, on the whole. Against this backdrop, compute PSNR and SSIM between the FPGA output and the associated software output when implementation fidelity is, at least in tenet, the question; truly, if gauging visual distortion relative to the original input, label that as a separate assessment, because the algorithm is anticipated to change the image, broadly speaking.

Algorithm	AG	RMS contrast	PSNR (dB)	MSE
Grayscale	5.39	12.76	7.6	11,454
Sobel	20.23	65.6	7.8	10,797
Gaussian	3.86	56.8	21	517
Unsharp masking	8.95	65.4	18.0	1,031

8.4.1 Metric definitions

Mean Squared Error (MSE) and Peak Signal-to-Noise Ratio (PSNR)

Mean Squared Error (MSE) and Peak Signal-to-Noise Ratio (PSNR) are applied to gauge the fidelity of the conducted image relative to the original source image, in actual application; indeed, in this project, the source and processed pixel values are contrasted for the estimation of the deviation introduced by image processing, in real-world scenarios.

The Mean Squared Error is specified as

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(X_i - Y_i)^2$$

where (X_i) represents the source-image pixel value, (Y_i) represents the matching processed-image pixel value, and (N) is the total number of pixels assessed, in many respects; and thus, a smaller average squared pixel difference between the two images is signaled by a lower MSE, in actual application.

The Peak Signal-to-Noise Ratio is commonly specified as

$$\mathrm{PSNR} = 10\log_{10}\left(\frac{MAX^2}{\mathrm{MSE}}\right)$$

For 8-bit grayscale images, the maximum pixel intensity is (MAX=255), giving

$$\text{PSNR} = 10\log_{10}\left(\frac{255^2}{\text{MSE}}\right) \text{ dB}$$

Greater pixel-level similarity between the source and handled images is being broadly implied by a higher PSNR, in actual application; and thus, when MSE is zero, equivalent pixel values and the theoretical PSNR are had by the images is infinite.
The suggested FPGA system encompasses a PSNR estimation unit outlined in psnr_calc.sv; in fact, pixel differences are ran by the unit, accumulates squared departures, assessments the mean squared error, and computes an encoded PSNR result applying fixed-point arithmetic, all things considered. In parallel with this, the hardware output representation, including its scaling, invalid-result indication, and zero-MSE sentinel handling, should be construed as stated by the operationalized RTL; these metrics quantitatively are basing for judging the image-processing operations conducted by the framework, in actual application.

8.5 Needed indication before final submission

- Quartus compilation and resource-utilisation report for the exact DE2-115 FPGA device.
- Clock constraints and complete static timing analysis, covering setup, hold and Fmax evidence.
- Power analysis report or recorded board-level power measurement, with premises.
- Embodied original and ran images for the four principal algorithms.
- A software-reference script and saved output images for all judged operations.
- Cycle counts and wall-clock timing per algorithm, plus a recorded definition of frame rate and latency, in real-world scenarios.
- PSNR/SSIM, entropy, average gradient, RMS contrast, maximum difference, and exact-match percentage from the same aligned image arrays, in real-world scenarios.
- A recorded memory map proving the source/destination and display-buffer storage paths do not conflict or wrap unexpectedly, at least in tenet, in real-world scenarios.

9. CONCLUSION

Taken together, simultaneously, VGA timing and RGB444 storage are implemented for the visible panes by the display path; the camera path converts RGB565 input to RGB888

and produces a 128 × 128 image. Along these dimensions, a custom RV32I processor, an image-processing accelerator, an OV7670 capture path, image-memory interfaces, a PSNR estimator, and a VGA display path in a person FPGA-oriented RTL design is combined by the project, at least in law; to be precise, the deployment is supporting custom-instruction-based operation selection and a set of image-processing modes, with grayscale, Sobel, Gaussian and unsharp masking chosen as the principal methods for this report, in actual application.

The central advancement is the system integration and execution of the processor, accelerator workflow, rather than the invention of new image-processing algorithms, on the whole; more correctly, quantitative conclusions must be added only after these findings are measured. A complete check is remaining dependent on final synthesis/timing reports, repeatable hardware tests, software-reference comparisons, and resolution of the memory-capacity/address-map issue summarized in the design chapter, on the whole.

9.1 Determined Results

Result area	Current proof from supplied RTL	Final report action
Processor and IPU integration	RTL modules and custom instruction path are, in practice, existent.	Attach regression/simulation or hardware indication.
Camera capture	OV7670 SCCB/capture RTL is mostly available; RGB565-to-RGB888 and crop/decimation logic are, in effect, mainly stipulated.	Insert a showed frame and substantiate timing/colour order.
Image-processing modes	Grayscale, Sobel, Gaussian and sharpening paths are, in essence, present in the IPU RTL, at least in tenet.	Compare each person output with a software reference.
PSNR	A divider-free estimator and IPU error accumulation are, in effect, existent.	Verify test vectors and distinguish source-relative distortion from reference-model exactness.
VGA display	640 × 480 timing and frame-display RTL are regularly available.	Insert board photograph and corroborate actual monitor output.
PPA / efficiency	Quartus reports for the four SoC builds are in Tables 8.1 to 8.3; no frame-rate or efficiency measurement exists yet, at least in tenet.	Populate from reports and reproducible measurements.

9.2 Future improvement/work

- Settle and document the full address map, source/destination capacity, and memory-access mediation.
- Add or formalise any confirmed software reference model for each person image-processing algorithm.
- Measure sustained FPS, per-algorithm latency, capture-to-display latency, and dropped frames.

- Optimise the IPU datapath through the pipelining, parallel arithmetic, or sharpened memory scheduling if measurement uncovers these are routinely holdups, at least empirically.
- Sharpen PSNR validation and, where advantageous, add an exact MSE/PSNR software analysis, alongside the hardware estimator, broadly speaking.
- Evaluate SSIM and other quality metrics on aligned reference images.
- Study larger images, external-memory use, multi-camera acquisition, or FPGA AI acceleration only as future work, at least in tenet; also, only after the current execution is measured, in many respects.

9.3 Person contribution from team members

The table below must be completed and authorized by this team; responsibilities must not be appointed by inference, to a significant degree; more correctly, two team members are listed by the supplied project roster but does not establish their person technical responsibilities, at least in law, in actual application.

Team member	Verified person contribution	Indication / confirmation
PANDALANENI PUJITH SAI VENKAT	[TEAM TO COMPLETE]	[CONFIRM]
RAGHAVA REDDY BOMMINENI	[TEAM TO COMPLETE]	[CONFIRM]

10. SOCIAL AND ENVIRONMENTAL IMPACT

The direct outcome of the project is the educational FPGA prototype that exposes embedded image acquisition, processor control, hardware acceleration, and reveal integration, at least in tenet; more exactly, all all those capabilities are relevant to learning about embedded vision systems and could (to the meaningful extent) base for future inspection or monitoring applications, in many respects. In light of this dynamic, against this backdrop, the put forward design should not be embodied as one further corroborated safety, medical, security, or AI recognition system, all things considered.
Environmental impact should be evaluated proportionately. Taken together, the design is systematically reusing any FPGA development board and camera module, which may (at least in theory) lessen the need for a second custom PCB during the prototyping, to one significant degree; and thus, potential refinements include lowering unnecessary frame writes, optimising memory transfers, and picking processing quality/resolution as stated by application needs, all things considered. Still no energy-saving claim is explained

without measuring power, under a stipulated workload and verifying it against a manifestly recommended alternative in this broader setting, at least for now.

11. COST ANALYSIS

Laboratory-owned equipment from costs that are needing confirmation is being separated by the following table, in many respects; and thus, against this backdrop, no authenticated purchase prices or invoices were supplied with the source files; in turn, enter actual costs and spot the price source before final submission, broadly speaking.

Item	Quantity	Unit cost	Total	Owned / purchased	Evidence/source
DE2-115 FPGA development board	1	[ENTER]	[ENTER]	[LAB-OWNED / PURCHASED]	[INVOICE / LAB INVENTORY]
OV7670 camera module	[ENTER]	[ENTER]	[ENTER]	[ENTER]	[INVOICE / ESTIMATE SOURCE]
VGA monitor and cable	[ENTER]	[ENTER]	[ENTER]	[ENTER]	[LAB-OWNED / PURCHASED]
Computer/workstation	1	[ENTER or N/A]	[ENTER]	[LAB-OWNED / PURCHASED]	[COSTING BASIS]
Software licences	[ENTER]	[ENTER or N/A]	[ENTER]	[ACADEMIC / PAID]	[VERSION / LICENCE]
Miscellaneous wires/adapters	[ENTER]	[ENTER]	[ENTER]	[ENTER]	[RECEIPTS]
Total project expenditure			[CALCULATE]		

In this setting, existing laboratory assets, report them separately as prepared rather than introducing their full replacement cost as newly incurred project expenditure, if the board and monitor were, in many respects; state the costing method employed, on the whole.

12. PROJECT OUTCOME PUBLICATION/PATENT

Publication or patent status was not established by the supplied files, and pick and complete the fitting statement: (a) no paper or patent has been submitted; (b) a manuscript is in preparation; (c) a manuscript has been submitted; before submission, at least for now. Viewed through this lens, within this framework, do not sketch an intended future submission as a completed publication, at least for now. Include verifiable bibliographic or application details only when supported by documentation; to be precise, still (d) a publication/patent has been accepted or granted, to a significant degree.

13. REFERENCES

[1] E. Gholizadehazari, T. Ayhan, and B. Ors, “An FPGA implementation of a RISC-V based SoC system for image processing applications,” in Proc. 29th IEEE Conf. Signal Process. Commun. Appl. Against this backdrop, (SIU), 2021, doi, specifically 10.1109/SIU53274.2021.9477998.

[2] H. Wang, D. Li, and T. Isshiki, “Reconfigurable CNN accelerator embedded in instruction extended RISC-V core,” in Proc. 6th Int. Conf. Electron. Technol. In this broader context, (ICET), 2023, doi, such as 10.1109/ICET58432.2023.10211513. Stated separately, [Bibliographic details supplied by project team; corroborate DOI/title against publisher before submission.]

[3] S. Wang et al. “Optimizing CNN computation using RISC-V custom instruction sets for edge platforms,” IEEE Trans Comput vol. 73, no. 5, pp. Taken together, 1371-1384, 2024, doi, like 10.1109/TC.2024.3362060.

[4] Q. Liu, W. Yin, and K. Li, “Image preprocessing acceleration method based on RISC-V vector extension,” J. Beijing Univ Aeronaut Astronaut. 1074-1084, 2025, doi, spanning 10.13700/j.bh.1001-5965.2023.0208. vol. 51, no. 4, pp.

[5] S. S. Kalagi et al., “FPGA validated RISC-V system on chip with a custom systolic array accelerator for edge AI inference,” Discover Artif. no. 883, 2026, doi, like 10.1007/s44163-026-02003-7. Building on this finding, intell., vol. 6, art.

[6] A. Waterman and K. Asanović, Eds., The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture. RISC-V International, version 20240411, Apr. Building on this observation, 2024. [Online]. Available: https://docs.riscv.org/reference/isa/v20240411/unpriv/unpriv-index.html

[7] D. A. Patterson and J. L. Hennessy, Computer Organization and Design RISC-V Edition: The Hardware/Software Interface. Along these dimensions, within this framework, cambridge, MA, USA: Morgan Kaufmann, 2017.

[8] S. L. Harris and D. M. Harris, Digital Design and Computer Architecture: RISC-V Edition, and building on this note, cambridge, MA, USA: Morgan Kaufmann, 2021.

[9] Terasic Inc., “DE2-115 FPGA Development and Education Kit,” product documentation. Viewed through this lens, [Online]. Available: https://www.terasic.com.tw/cgi-bin/page/archive.pl?CategoryNo=139&Language=English&No=502

[10] Terasic Inc., DE2-115 User Manual, ver. 2.3, 2017. [Online]. Taken together, available: https://www.terasic.com.tw/wiki/images/f/ff/DE2_115_User_manual_2013.pdf

[11] OmniVision Technologies, “OV7670/OV7171 CMOS VGA (640×480) CameraChip Sensor Data Sheet,” version 1.01, Jul. 2005. [Online]. Available: https://www.olimex.com/Products/Components/Camera/CAMERA-OV7670/resources/OV7670.pdf

[12] Intel Corporation, Cyclone IV Device Handbook. From this perspective, viewed through this lens, intel FPGA technical documentation. [Online]. Available: https://www.intel.com/content/www/us/en/products/details/fpga/cyclone/iv.html

[13] Intel Corporation, Quartus Prime Pro Edition User Guide, specifically Design Compilation. Intel FPGA documentation. [Online]. Available: https://www.intel.com/content/www/us/en/software/programmable/quartus-prime/overview.html

[14] IEEE, IEEE Standard for SystemVerilog, Unified Hardware Design, Specification and Verification Language, IEEE Std 1800, in real-world scenarios, and apply the edition applicable to the project toolchain.

[15] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed. In parallel with this, new York, NY, USA: Pearson, 2018.

[16] I. Sobel and G. Feldman, “A 3×3 isotropic gradient operator for image processing,” presented at the Stanford Artificial Intelligence Project, 1968.

[17] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” IEEE Trans. Image Process. vol. 13, no. 4, pp. Along these dimensions, 600-612, Apr. 2004, doi: 10.1109/TIP.2003.819861.

[18] A. Hore and D. Ziou, “Image quality metrics: PSNR vs. SSIM,” in Proc. 20th Int. Conf. Pattern Recognit. (ICPR), 2010, pp. 2366-2369, doi: 10.1109/ICPR.2010.579.

[19] J. Canny, “A computational approach to edge detection,” IEEE Trans. Pattern Anal. Mach. Viewed through this lens, intell. vol. PAMI-8, no. 6, pp. 679-698, Nov. 1986, doi: 10.1109/TPAMI.1986.4767851.

[20] D. Marr and E. Hildreth, “Theory of edge detection,” Proc. R. Soc. Lond. 187-217, 1980, doi, namely 10.1098/rspb.1980.0020. In light of this dynamic, building on this finding, b, vol. 207, no. 1167, pp.

[21] A. K. Jain, Fundamentals of Digital Image Processing; thus, in parallel with this, englewood Cliffs, NJ, USA: Prentice Hall, 1989.

[22] R. C. Gonzalez, R. E. Woods, and S. L. Eddins, Digital Image Processing Using MATLAB, 2nd ed. Building on this observation, gatesmark Publishing, 2009.

[23] T. Acharya and A. K. Ray, Image Processing: Principles and Applications, and viewed through this lens, hoboken, NJ, USA: Wiley, 2005.

[24] T. M. Cover and J. A. Thomas, Elements of Information Theory, 2nd ed; thus, building on this finding, hoboken, NJ, USA: Wiley, 2006.

[25] A. Mittal, R. Soundararajan, and A. C. Bovik, “Making a ‘completely blind’ image quality analyzer,” IEEE Signal Process. Lett. vol. 20, no. 3, pp. 209-212, Mar. 2013, doi: 10.1109/LSP.2012.2227726.

[26] A. Mittal, A. K. Moorthy, and A. C. Bovik, “No-reference image quality assessment in the spatial domain,” IEEE Trans. Image Process. Viewed through this lens, vol. 21, no. 12, pp. 4695-4708, Dec. Building on this observation, 2012, doi: 10.1109/TIP.2012.2214050.

[28] RISC-V International, “RISC-V Technical Specifications and Documentation,” [Online]. Available: https://docs.riscv.org/.

[29] Intel Corporation, “FPGA design software and Quartus Prime documentation,” [Online]. Available: https://www.intel.com/content/www/us/en/software/programmable/quartus-prime/overview.html

[30] D. A. Patterson and J. L. Hennessy, Computer Architecture: A Quantitative Approach, 6th ed; thus, taken together, cambridge, MA, USA: Morgan Kaufmann, 2017.

APPENDIX A, FIGURE AND TABLE REGISTER

Employ this register to sustain clear numbering after inserting the actual figures; after all content is finalised, update the list of figures and list of tables in Word, in real-world scenarios.

Figure	Needed proof / content	Status
1.1	Project scope and end-to-end data path	[PLACEHOLDER]
4.1	Top-level RISC-V SoC architecture	[PLACEHOLDER]
4.2	Five-stage RV32I processor pipeline	[PLACEHOLDER]
4.3	OV7670 capture and RGB conversion flow	[PLACEHOLDER]
4.4	IPU finite-state-machine processing flow	[PLACEHOLDER]
4.5	Grayscale, Sobel, Gaussian and unsharp-masking kernels	[PLACEHOLDER]
4.6	PSNR estimation data path	[PLACEHOLDER]
5.1	DE2-115 hardware setup	[PLACEHOLDER]
5.2	VGA source and processed image layout	[PLACEHOLDER]
8.1	Quartus resource utilisation report	[PLACEHOLDER]
8.2	Timing investigation report	[PLACEHOLDER]
8.3	Original and handled images: grayscale	[PLACEHOLDER]
8.4	Original and handled images: Sobel	[PLACEHOLDER]
8.5	Original and handled images: Gaussian	[PLACEHOLDER]
8.6	Original and handled images - unsharp masking	[PLACEHOLDER]
8.7	Image-quality and throughput comparison	[PLACEHOLDER]

Table	Content	Status
4.1	Key design parameters	[DRAFT / UPDATE]
4.2	Principal image-processing algorithms	[DRAFT / UPDATE]
6.1	Hardware and software tools	[DRAFT / UPDATE]
7.1	Schedule and milestone status	[DRAFT / UPDATE]
8.1	FPGA resource utilisation	[DRAFT / UPDATE]
8.2	Timing and power findings	[DRAFT / UPDATE]
8.3	Per-algorithm speed results	[DRAFT / UPDATE]
8.4	Image-quality findings	[DRAFT / UPDATE]
8.5	Efficiency results	[DRAFT / UPDATE]
9.1	Obtained results and evidence status	[DRAFT / UPDATE]
11.1	Project cost investigation	[DRAFT / UPDATE]
B.1	RTL module inventory	[DRAFT / UPDATE]
C.1	Experimental checklist	[DRAFT / UPDATE]

APPENDIX B, RTL MODULE INVENTORY

That inventory is systematically grounded in the supplied RTL filenames. The uploaded directory contains repeated copies with numbered suffixes; the final project repository should identify one canonical version of each module and ensure that the files compiled by Quartus are the versions represented in this thesis.

RTL file / module	Purpose
riscv_image_soc_top_de2115.sv	Board-level top integration; switch decode, clock/reset, camera GPIO, VGA outputs, processing/frame counters.
rv32i_core_fpga.sv	Core wrapper connecting instruction ROM, pipeline/datapath, data memory and IPU.
datapath.sv	Processor datapath integration.
fetch_stage.sv	Instruction fetch stage.
decode_stage.sv	Instruction decode and operand/control stage.
execute_stage.sv	ALU/branch/execute stage.
writeback_stage.sv	Register writeback stage.
program_counter.sv	Program counter state.
register_file.sv	Integer register file.
imm_generator.sv	Immediate generation for supported instruction formats.
main_control.sv	Central instruction control.
alu_control.sv	ALU operation decode.
alu.sv	Arithmetic/logic operations.
adder.sv	Adder primitive.
branch_unit.sv	Branch condition evaluation.
if_id_reg.sv	IF/ID pipeline register.
id_ex_reg.sv	ID/EX pipeline register.
ex_mem_reg.sv	EX/MEM pipeline register.
mem_wb_reg.sv	MEM/WB pipeline register.
hazard_detection_unit.sv	Hazard detection/stall logic.
forwarding_unit.sv	Operand forwarding logic.
instruction_memory_fpga.sv	Instruction source and operation-specific custom instruction selection.
data_memory_fpga.sv	Byte-lane memory, CPU/IPU/host paths and status interface.
ipu_fpga.sv	IPU FSM, pixel operations, memory writes, counters and error accumulation.
psnr_calc.sv	Lookup-based divider-free PSNR estimate.
ov7670_sccb_init.sv	Camera configuration over SCCB.
ov7670_capture_128_rgb565_to_rgb888.sv	Camera capture, RGB565-to-RGB888, crop/downsample, byte write interfaces.
vga_640x480.sv	VGA sync and active-area timing.
vga_frame_display.sv	Point to frame RAM, source/processed composition, RGB444 conversion and overlays.

APPENDIX C, EXPERIMENTAL CHECKLIST

Final verification item	**Done**
Confirm Quartus top-level module and exact FPGA part	[ ]
Resolve memory capacity/address-map overlap risk	[ ]
Verify source and destination buffer read/write paths	[ ]
Represent a known OV7670 frame and substantiate RGB byte order	[ ]
Save output image for each principal algorithm	[ ]
Run software reference with matching rounding and border rules	[ ]
Confirm PSNR estimator with known MSE vectors	[ ]
Export LUT/FF/RAM/DSP usage report	[ ]
Export setup/hold timing and Fmax report	[ ]
Export/measure static and dynamic, with a center on total power	[ ]
Measure processing cycles and wall-clock time per algorithm	[ ]
FPS, throughput, latency were sustained by measure and dropped frames	[ ]
Calculate entropy, AG, RMS contrast, PSNR, SSIM and pixel accuracy	[ ]
Evaluate efficiency metrics from clear power/FPS data	[ ]
Complete cost analysis with proof	[ ]
Substantiate each team member's advancement	[ ]
Corroborate at least 20 IEEE-style references and all in-text citations	[ ]
Update table of contents, figure/table lists and pagination	[ ]
Remove all placeholders that have been settled	[ ]

End of draft. All bracketed placeholders and source-verification notes must be settled, before this document is submitted as any final thesis, in many respects.